

Full-Stack Developer Technical Challenge — "InboxZero"

An AI batch triage service. Submit a pile of messages, a **background worker** processes them with AI, and the UI tracks progress in real time. The focus: **asynchronous job processing done right**.

1. Overview

InboxZero ingests a batch of text items (think: a dump of support emails, customer messages, or feedback) and, **in the background**, uses AI to process each one: classify it (e.g. `bug` / `billing` / `feature-request` / `spam`), score its **priority** and **sentiment**, write a one-line **summary**, and draft a **suggested reply**. The user uploads a batch, gets a job, and watches the items move from `queued` → `processing` → `done` while results stream in.

The product is simple. The point is **not** the triage idea — it's to see whether you can build **asynchronous, queue-based processing** correctly: a real job queue, a separate **worker** process/container, status tracking, **retries**, **idempotency**, and a UI that reflects progress without blocking.

-  **Time box: 3 days.** The hard part is the async plumbing, not the AI. Keep the AI prompt trivial; make the queue/worker/retry story solid.
-  **Design / UI / UX does NOT matter.** A table of items with status badges is plenty. Spend your time on the background pipeline, not pixels.

2. The Purpose (Why this challenge exists)

We want to evaluate whether you can:

- Offload slow work to a **background worker** instead of blocking the request thread.
- Use a **real job queue** (Celery+Redis, RQ, BullMQ, pg-boss, or similar) with a **separate worker container**.
- Track **job + per-item status** in Postgres and expose it to the UI.
- Handle **failure, retries, and partial completion** gracefully (one bad item must not sink the batch).
- Make processing **idempotent** (re-running / retrying an item doesn't duplicate or corrupt results).
- Keep it **multi-tenant safe** — a user only sees their own jobs.
- Ship it: **Docker** locally and a **live Vercel** frontend.

3. Allowed Tech Stack

Layer	Allowed choices
Backend	Python + FastAPI , or Node.js + Express.js (TypeScript)
Worker / Queue	Celery + Redis or RQ (Python); BullMQ, pg-boss, or similar (TypeScript). A real queue + separate worker process is required.
Frontend	React + Next.js (TypeScript), React Query for data fetching
Database	PostgreSQL (required)
Migrations	Alembic (if Python) or a TypeScript migration tool (Prisma, Drizzle, Knex, TypeORM)
Auth	JWT (simple — email/password → token)
AI	One of: Anthropic Claude , OpenAI , Google Gemini , or Groq — via your own API key
Containers	Docker (Dockerfile + docker-compose: API + worker + Postgres + Redis/broker)

Deployment	Frontend on Vercel (required, live URL)
------------	--

The backend may run locally or on the cloud. **If the backend runs locally**, you must expose it to your live Vercel frontend (see §6 — "Proxy / tunnel requirement"). We need to open your Vercel URL and have a working app, even if your backend (and its worker) are on your laptop at demo time.

4. Functional Requirements (Scope)

Keep it to exactly this. No more is needed.

Auth

- ☐ **Register** with email + password.
- ☐ **Login** returns a JWT.
- ☐ Protected endpoints reject requests without a valid token (401).
- ☐ A user can only see and act on **their own** jobs and items.

Submitting a batch

- ☐ **Create a job** by submitting a batch of text items (paste a list, or upload a `.csv` / `.txt` — your choice; a textarea of newline-separated items is acceptable).
- ☐ The API returns **immediately** with a job id and `queued` status — it must **not** block while the AI runs.
- ☐ Each item is enqueued for background processing.

Background processing (the core feature)

- ☐ A **separate worker** picks up items from the queue and processes each with AI: `category` , `priority` , `sentiment` , `summary` , `suggested_reply` .
- ☐ **Per-item status**: `queued` → `processing` → `done` / `failed` .
- ☐ **Retries**: a transient failure (e.g. AI timeout) is retried up to N times with backoff; after that the item is marked `failed` with an error — the **rest of the batch keeps going**.
- ☐ **Idempotent**: retrying or re-processing an item must not duplicate results or double-apply anything.
- ☐ **Job-level rollup**: the job reports counts (`queued` / `processing` / `done` / `failed`) and an overall status (`processing` until all items are terminal, then `completed`).

Frontend (Next.js + React Query)

- ☐ Login / register screen.
- ☐ A screen to submit a new batch.
- ☐ A job view: a table of items with their **status** and (when done) their AI results.
- ☐ Progress updates **live** as the worker advances — use React Query **polling** (`refetchInterval`) that **stops once the job is complete**, or WebSocket/SSE if you prefer.
- ☐ After submitting a batch, the jobs list **updates automatically** via React Query **cache invalidation** — not a full page reload.
- ☐ A way to **retry failed items** (re-enqueue) that correctly updates state.

5. Suggested Data Model

You may adjust, but something like:

users	
id	(pk)

```

email          (unique)
password_hash
created_at

jobs
  id            (pk)
  user_id       (fk -> users.id)
  status        (processing | completed)
  total_items   (int)
  created_at

items
  id            (pk)
  job_id        (fk -> jobs.id)
  user_id       (fk -> users.id)    -- denormalized for safe scoping
  input_text    (text)
  status        (queued | processing | done | failed)
  attempts     (int, default 0)
  -- AI results:
  category      (text, nullable)
  priority      (text/int, nullable)
  sentiment     (text, nullable)
  summary       (text, nullable)
  suggested_reply (text, nullable)
  error         (text, nullable)
  updated_at

```

✓ We must be able to see **migration file(s)** that create these tables (Alembic revision, or your TS migration tool's generated migration). Do **not** auto-create the schema silently at runtime only — show us the migration.

6. Deployment Requirements (important)

1. Frontend **MUST** be deployed to Vercel with a public URL we can open.
2. The deployed frontend must talk to a **real backend + worker + Postgres + broker**.
3. The backend location is your choice:
 - **Cloud** (Railway, Render, Fly.io, AWS, etc.) — straightforward, or
 - **Local** — then you must put it behind a **proxy / tunnel** (e.g. ngrok, cloudflared, Next.js rewrites/proxy) so the live Vercel site reaches it. Document exactly how to start the tunnel.
4. The database and broker must be reachable by whatever runs the backend and worker (hosted Postgres/Redis like Neon/Upstash/Railway is fine, or local if the backend is local).

The reason for the proxy requirement: we want to confirm you understand how a deployed frontend reaches a backend across origins (CORS, base URLs, env-based API URLs, tunneling). Note the worker does **not** need to be public — only the API.

7. Containerization Requirements

- ☐ A **Dockerfile** for the backend (and the worker, if separate image).
- ☐ A **docker-compose.yml** that brings up **API + worker + Postgres + broker (Redis or your queue's broker)** with a single command. The worker must be its **own service** in compose.
- ☐ Running migrations should be a documented step (e.g. `docker compose run api <migrate command>`).
- ☐ We should be able to clone the repo, set env vars, and run `docker compose up` to get a working local stack where submitting a batch is processed by the worker.

8. What You Must Deliver

1. A **Git repository** (GitHub/GitLab) containing all code.
2. A **README.md** with:
 - Short description.
 - Tech choices — including **which queue/broker** you used.
 - A short note on your **retry & idempotency** strategy.
 - **How to run locally** (env vars, `docker compose up`, how to run migrations, how the worker starts).
 - **The live Vercel URL.**
 - If backend is local: exact commands to start the tunnel/proxy.
 - A test account, or registration instructions.
 - Which AI provider you used.
3. A **.env.example** listing every required environment variable (no real secrets).
4. **Migration file(s)** committed to the repo.
5. A **demo video** (2–5 min) — showing: register/login, submitting a batch, the API returning **instantly**, items moving through statuses **live**, a **failed item** (you can simulate one) and **retrying** it, and the final job rollup.

⚠️ Do **not** commit real API keys or secrets. Committed secrets are an automatic red flag.

9. What Will Be Tested / Evaluated

Area	What we check
Async correctness	Submit endpoint returns immediately; work happens in a separate worker , not in the request.
Queue & worker	A real queue + worker service in compose; items are actually consumed off the queue.
Status tracking	Per-item and job-level status are accurate and converge to a terminal state.
Retries & resilience	Transient failures retried with backoff; permanent failures isolated; one bad item doesn't sink the batch.
Idempotency	Retrying/re-running doesn't duplicate or corrupt results.
Auth	JWT; a user can't see or retry another user's jobs/items.
React Query	Live progress via polling that stops when done (or WS/SSE); jobs list invalidated after submit.
Database & Migrations	Postgres + real migration files; sensible schema.
Docker	<code>docker compose up</code> brings up API + worker + DB + broker and works.
Deployment	Vercel URL live and functional; backend reachable (cloud or via documented tunnel).
Code quality	Structure, naming, error handling, no secrets, clear README.
Communication	README clarity + demo video.

10. Expected Outcomes (Definition of Done)

- ☒ I can open your **Vercel URL** and register/log in.
- ☒ I can **submit a batch** and the API responds **immediately** with a job.
- ☒ Items are processed by a **separate worker** and I watch statuses advance **live**.
- ☒ A **failed item** is isolated, retried, and I can **manually retry** it; the batch still completes.

- ☒ Re-processing does **not** duplicate results (idempotent).
- ☒ The jobs list **updates without a manual refresh** after I submit; polling **stops** when the job is complete.
- ☒ A second user sees **none** of the first user's jobs.
- ☒ I can `git clone`, set `.env`, run `docker compose up` (API + worker + DB + broker), run the migration, and it works locally.
- ☒ **Migration file(s)** and `.env.example` are committed.
- ☒ There is a **demo video** linked in the README.
- ☒ No secrets are committed.

11. Scoring Guide (100 pts)

Criterion	Points
Async submit (non-blocking) + real queue + separate worker	22
Per-item & job status tracking, converging to terminal state	15
Retries / backoff / failure isolation	15
Idempotency	10
Auth (JWT, per-user scoping)	10
Frontend + React Query live progress (polling stops when done) + invalidation	12
Postgres + migrations	6
Docker compose (API + worker + DB + broker)	6
Vercel deployment live (+ proxy if local backend)	4
Bonus: WebSocket/SSE push instead of polling, rate-limiting AI calls, dead-letter queue, tests, CI	+up to 10

12. Tips & Boundaries

- **Don't over-engineer the AI.** One cheap classification prompt per item is enough. The grade is in the plumbing.
- **Keep batches small** (e.g. cap at 20–50 items) to stay within timeouts and API rate limits.
- **Make idempotency explicit:** key results by item id, use status guards / unique constraints so a re-run is a no-op or a clean overwrite.
- **Simulate failures** for the demo (e.g. an item containing the word `FAIL` always throws) so you can show retries — document this.
- **Rate-limit** your AI calls so a batch doesn't trip provider limits (a small concurrency cap in the worker is plenty).
- **Stop polling** when the job is terminal — leaving a `refetchInterval` running forever is a common mistake we look for.
- Commit early and often.
- If you run out of time: a **bulletproof small queue** beats a flaky big one.

13. Submission Checklist (paste into your README)

[] Repo link

[] Live Vercel URL

[] Backend location (cloud URL, or "local + tunnel" with start command)

[] Test account or registration instructions

[] AI provider used

[] Queue/broker used + retry & idempotency strategy documented

```
[ ] .env.example committed
[ ] Migration file(s) committed
[ ] docker compose up brings up API + worker + DB + broker
[ ] Demo video link
[ ] No secrets committed
```

Good luck — we're excited to see what you build! 🚀

Property of Efsora Labs.